

One question before we keep optimising the zip path

Hi DevOps,

Perforce (Robert Cowham) has just shared a working alternative to `p4 zip` — a small bash wrapper around `p4 push` that pushes one changelist at a time using a remote spec, with a LastPush counter so it's fully resumable. In his experience it is **more reliable than `p4 zip`** for depots with renames and dirty history, and it sidesteps both of the walls we have hit so far (the 100,000-line DepotMap cap and the orphan-move counterpart issue).

The trade-off: `p4 push` is a **server-to-server protocol**, so it requires the destination `p4d` to be up and reachable from `illin2343` on the network. `p4 zip` produces a portable archive and does not.

Question for you: is the destination `p4d` already provisioned and reachable from `illin2343`?

- **If yes** — we strongly recommend pivoting to the `p4 push` path. We will layer our auto-exclude-orphan logic on top of Robert's wrapper so individual CLs that contain an obliterated move counterpart skip cleanly instead of stalling the migration. Expected throughput per Robert: 10–40 changes/minute. Single-command run, fully resumable.
- **If no** — we continue the `p4 zip` + auto-exclusion path we are already on (`option-b-clone-and-sanitize` branch on GitHub). It works, it is just more friction.

Either way our team owns the workaround end-to-end; Perforce do not need to ship a fix.

Please confirm topology and we will adjust within a few hours.

Thanks, [your name] ESL / Perforce reseller